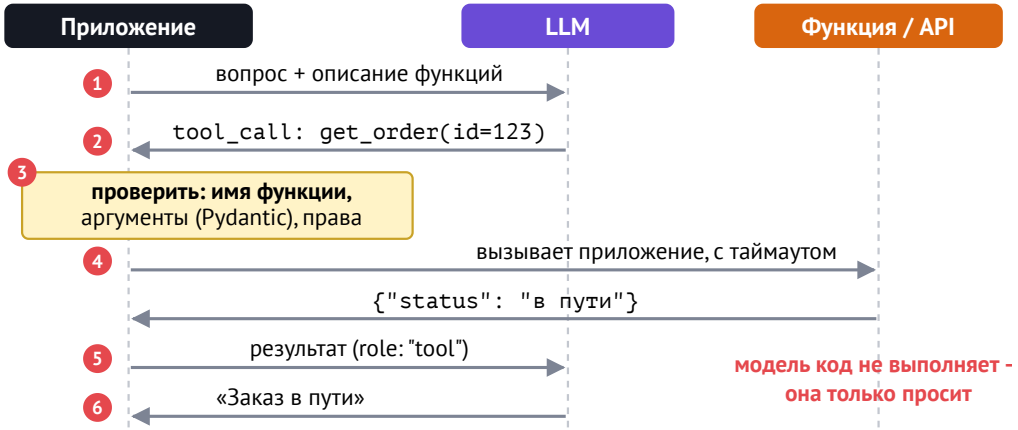
Tool calling и агенты

01 Tool calling



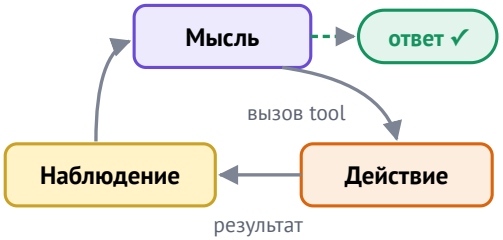
Модель вместо текста возвращает **имя функции и аргументы в JSON** по описанию инструментов. Вызов, валидация, права и таймауты — на приложении. Результат возвращают модели, и она пишет ответ. Один ответ может содержать несколько tool_call.

02 Агенты: ReAct и Plan-and-Execute

Обычный вызов LLM
один запрос → один ответ текстом; шаги задаёт код

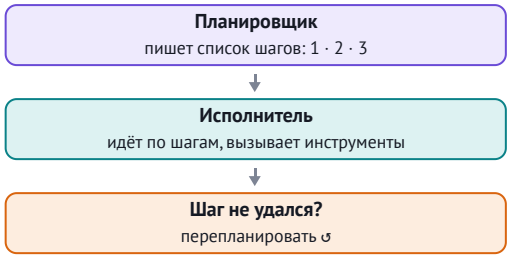
Агент — LLM **в цикле с инструментами**: сама выбирает следующий шаг, действует, смотрит на результат и продолжает, пока не достигнет цели

`REACT: ДУМАЙ → ДЕЙСТВУЙ → СМОТРИ`



`Мысль: нужен статус → get_order(123) → «в пути» → ответ`

`PLAN-AND-EXECUTE: СНАЧАЛА ПЛАН`



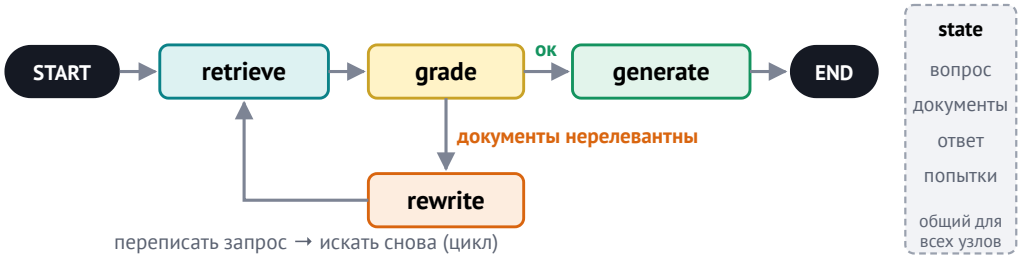
Для длинных задач: план видно и можно проверить заранее.

Ограничители обязательны: лимит шагов и токенов, таймауты, подтверждение опасных действий человеком (оплата, удаление), логи каждого шага.

03 LangChain и LangGraph

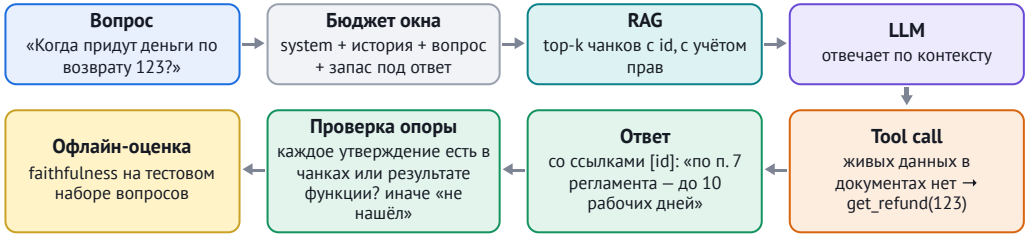
LangChain — готовые блоки: модели, промпты, загрузчики, сплиттеры, ретриверы, цепочки. Быстро собрать RAG и сменить провайдера.

LangGraph — агент как **граф**: узлы, общий **state**, условные переходы, **циклы**, сохранение состояния, человек в контуре.



Простой линейный сценарий — **обычный код**. Ветвления, циклы, долгие агенты с паузой на человека — **LangGraph**.

04 Ассистент по документам + функция: как всё связано



1. **Токены:** окно и цена не дают положить всю базу знаний в промпт
2. **RAG** выбирает несколько релевантных кусков — только из того, что пользователю можно видеть
3. **Tool calling** — за живыми данными (статус, остатки); модель просит, приложение вызывает
4. **Опора на источник:** ссылки [id] на переданные чанки + офлайн-метрика **faithfulness** на тестовом наборе

05 Частые ошибки

- ~~«Модель сама вызывает функцию»~~ → только просит, вызывает приложение
- ~~«Агент — любой чат-бот»~~ → цикл с инструментами и решениями модели
- ~~«Чем больше чанков, тем лучше»~~ → шум и деньги
- ~~«RAG — это дообучение»~~ → подкладывает найденное в промпт
- ~~«Проверили на 5 вопросах»~~ → тестовый набор и метрики